

Plan d'action — Orchestrer Claude Code pour AI for Geopolitics

Laury Kibamba

30 juin 2026

Table of Contents

TL;DR

- **Cowork (cette session) et Claude Code partagent le même moteur et la même qualité de code.** Ce qui change, c'est l'environnement d'exécution.
- **Cowork = couche pensée/auteur** (recherche, design, documents, scaffolding, prototypes). **Claude Code (terminal) = couche exécution** (GPU, Docker, git, tests — le vrai dev).
- **Boucle standard par feature : Explore → Plan (plan mode) → Code (TDD) → Verify (tests + sous-agent reviewer) → Commit** (atomique, conventionnel).
- **Reste dans la phase courante** (P0→P7). Un sous-agent pour le bruit, des hooks comme garde-fous, des git worktrees pour le parallèle.

1. Cowork vs Claude Code : où faire quoi

Réponse franche à ta question : **oui, c'est le même Claude et la même capacité de code** — Cowork est bâti sur Claude Code et l'Agent SDK. La différence n'est pas l'intelligence, c'est **ce que l'environnement peut toucher**.

Capacité	Cowork (ici)	Claude Code (ton terminal)
Même modèle / qualité de code	Oui	Oui
Écrire des fichiers dans le dossier projet	Oui	Oui
Lancer du code	Sandbox Linux isolé (≈2 cœurs, ~4 Go, sans GPU)	Ta vraie machine (RTX 2060 Super, 32 Go)
GPU : nvidia-smi, Ollama, le service d'inférence	Non	Oui
Docker / Kubernetes réels	Non (sauf via MCP connecté)	Oui
Git local, pytest, ports, .env	Non	Oui
Plan mode, sous-agents,	Via cette UI	Nativement

hooks, slash commands

Idéal pour

recherche, design, docs,
scaffolding

**build / run / test, GPU,
infra**

Division du travail. Utilise **Cowork** comme co-pilote de réflexion (ce qu'on vient de faire : état de l'art, scaffolding, ce plan). Utilise **Claude Code dans ton terminal** pour tout ce qui s'exécute sur ta machine — c'est là que vit le projet, parce que ton GPU, Docker et git ne sont accessibles que là.

Nouveauté utile : des connecteurs MCP **Kubernetes** et **Windows** sont apparus dans cette session. Branchés, ils permettraient à Cowork d'agir sur un vrai cluster (kubectl) ou ta machine (PowerShell) — l'écart se réduit. Mais pour du dev soutenu, le terminal Claude Code reste la maison naturelle.

2. Les primitives d'orchestration de Claude Code

Primitive	À quoi ça sert ici
CLAUDE.md (déjà fait)	Mémoire projet lue à chaque session : archi, phases, principes. Garde-le lean .
Plan mode (Shift+Tab)	Lecture seule : Claude explore et propose un plan sans éditer. À valider avant tout code.
Sous-agents (.claude/agents/*.md)	Contexte isolé pour les tâches bruyantes (recherche, review) ; outils restreints ; modèle moins cher (Haiku) pour router.
Slash commands / Skills (.claude/commands/ → .claude/skills/)	Workflows répétables invoqués par /nom. Les commandes sont désormais fusionnées dans les skills (l'ancien format .claude/commands/ marche encore).
Hooks (déjà : format_lint)	Garde-fous déterministes (format, lint, tests) déclenchés par événement.
MCP (déjà : github / postgres / fs)	Outils externes ; côté projet, les outils-pays seront des serveurs MCP .
Git worktrees	Plusieurs sessions Claude en parallèle sur des branches isolées (1 worktree = 1 feature).
/clear + budget de réflexion	/clear entre deux tâches pour repartir propre ; « think hard » / « ultrathink » pour les décisions d'archi.
Headless (claude -p "...")	Scripter Claude en CI ou en batch (ex. évaluation RAG nocturne).

3. La boucle de travail standard (par feature)

Le workflow qui évite que Claude « code la mauvaise chose » :

1. **Explore** — « Lis core/ et simulation/, ne modifie rien. Résume comment le round engine appelle le moteur de conséquences. » (ou laisse le sous-agent *Explore* le faire).
2. **Plan** — passe en **plan mode**, demande un plan d'implémentation. **Tu valides** (ou tu corriges) avant la moindre édition.
3. **Code (TDD)** — « Écris d'abord les tests pytest pour ce comportement, puis l'implémentation jusqu'au vert. » Le hook ruff formate/lint à chaque édition.
4. **Verify** — lance les tests ; fais relire par un **sous-agent reviewer** contre la spec (cf. §5). Pour le GPU : profile (nvidia-smi, tokens/s) **avant** d'optimiser.
5. **Commit** — commit **atomique**, message conventionnel (feat:/fix:/test:...), petite PR. /clear puis feature suivante.

4. Plan par phase (P0 → P7)

Phase	Objectif	Où	Orchestration Claude Code	Definition of done
P0 Moteur déterministe	CountryState/ WorldState/ GeoEvent, round engine + conséquences, sans LLM	Cowork ou CC	Plan mode → TDD ; pur Python	Un round simulé de bout en bout, tests verts
P1 Agents LLM + service d'inférence	InferenceBack end + impl (llama-cpp- python), CountryAgent, sortie JSON validée (Pydantic)	CC (GPU)	Sous-agent <i>inference</i> , profilage VRAM, fallback parsing	6–8 agents décident en JSON valide ; tok/s mesurés
P2 Diplomatie	Messages bilatéraux, alliances, accept/refuse	CC	Plan mode ; tests d'intégration	Négociation visible + résumé public
P3 RAG	Chroma + BM25 + RRF + reranking + citations ; éval recall@k/MRR	CC	Sous-agent <i>rag- eval</i> en headless	Brief sourcé + métriques retrieval
P4 Données réelles	Ingestion World	Cowork (ingestion) + CC	Sous-agent <i>geo- researcher</i> ;	Pays construits depuis datasets,

	Bank/SIPRI/GD ELT → CountryState		data_governan ce.md	pas au prompt
P5 Interface	Dashboard (timeline, tensions, alliances, messages)	CC	—	Un round lisible dans l'UI
P6 Infra	Docker (image/service) → compose → kind/K8s	CC (+ MCP Kubernetes)	kubectl via MCP ; docs/deploym ent.md	Stack qui démarré en local, documentée
P7 MCP / distribué	Outils-pays en MCP, broker Redis	CC	Sous-agent <i>mcp-builder</i>	Agents qui appellent des outils MCP

5. Sous-agents & commands à créer (minimal)

Trois sous-agents suffisent pour démarrer. Exemple `.claude/agents/test-runner.md` :

```
---
name: test-runner
description: Lance pytest et corrige au minimum les tests qui échouent. À utiliser
après toute implémentation.
tools: Read, Edit, Bash
model: sonnet
---
```

Tu es un ingénieur QA. Lance la suite pytest, analyse les échecs, corrige le code ou les tests au strict nécessaire, relance jusqu'au vert. Ne change pas le périmètre fonctionnel.

`.claude/agents/geo-researcher.md` (recherche sourcée pour P4) :

```
---
name: geo-researcher
description: Recherche sourcée (chiffres, données) pour alimenter un
CountryState ou un scénario. PROACTIVEMENT pour toute donnée géopolitique.
tools: WebSearch, WebFetch, Read, Write
model: sonnet
---
```

Trouve des données récentes et CITE chaque source (URL). Renvoie un JSON conforme au schéma CountryState.
N'invente jamais un chiffre ; si introuvable, dis-le.

Un *code-reviewer* (tu as déjà le skill `engineering:code-review` à réutiliser). Côté commande, ex. `.claude/commands/commit.md` :

***description:** Crée un commit conventionnel à partir des changements en cours*

Analyse ``git diff --staged``, propose un message conventionnel (feat/fix/test/docs/refactor), puis exécute le commit. Garde-le atomique.

6. Git & CI

- **Branche par feature/phase** ; commits atomiques ; PRs petites.
- **CI minimale** (.github/workflows/ci.yml) qui rejoue tes garde-fous :

```
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install ruff pytest -e .
      - run: ruff check .
      - run: pytest -q
```

Ainsi les hooks (locaux) **et** la CI (distante) appliquent la même exigence : code propre + tests verts.

7. Garde-fous & anti-patterns

- **Ne laisse jamais Claude foncer dans le code** : plan mode d'abord, validation ensuite.
- **CLAUDE.md lean** : s'il gonfle, déplace le détail vers docs/.
- **/clear entre les tâches** : le contexte sale provoque des erreurs.
- **Mesure avant d'optimiser** (VRAM, tok/s) — surtout sur 8 Go.
- **Reste dans la phase** : pas de RAG ni K8s en P1. Le danger n°1 est l'explosion de complexité.
- **Aucun secret dans le repo** ; .env + variables d'environnement.

8. Checklist de démarrage (ordonnée)

1. Installer Claude Code dans le terminal ; ouvrir le dossier projet (il lit déjà CLAUDE.md).
2. pip install ruff pytest ; Docker Desktop ; (optionnel) Ollama + un modèle 7–8B Q4.
3. Exporter GITHUB_PERSONAL_ACCESS_TOKEN ; vérifier l'approbation des serveurs .mcp.json.
4. Créer les 2–3 sous-agents (\$5) et la commande /commit.

5. git init + premier commit du scaffold.
 6. **Lancer P0** : plan mode → « propose les schémas Pydantic + le round engine déterministe, avec tests d'abord ».
 7. Boucle §3 jusqu'à la fin de P0, puis passer à P1 **sur ta machine** (GPU).
-

Rappel : Cowork pour penser/écrire/scaffolder, Claude Code (terminal) pour exécuter sur ton matériel. Ce document vit dans docs/ et peut être lu par Claude Code.